

Hermes Agent Skill Set

表示、索引、查询与自进化机制详细源码整合

专业学习文档 | 详细专业版

核心结论

Hermes 的 Skill 不是一条 Prompt、不是向量库条目，也不是模型参数。它更接近“文件夹式程序性记忆”：以 SKILL.md 为入口，附带 references/templates/scripts/assets，通过系统提示中的轻量元数据目录暴露给 LLM，再由 skill_view 渐进加载完整内容。所谓自进化，本质是 Background Review、skill_manage、usage sidecar 与 Curator 共同驱动的文件级知识库演进，而不是在线梯度训练。

输入材料：D1《Hermes Agent Skill Set 源码学习文档》；D2《Hermes Skill 自进化完整源码机制解析》

整合目标：去重、补全机制链路、提升源码级解释密度，并形成可用于组会/面试/二次研究的结构化文档。

目录

- 0. 阅读说明与整合原则
 - 1. 一句话总览：Hermes Skill 的系统位置
 - 2. 概念边界：Skill、Memory、RAG、Self-Evolution
 - 3. Skill 的文件级表示与所有权边界
 - 4. Index 构建：扫描、过滤、缓存与上下文注入
 - 5. 查询与加载：skills_list、skill_view 与渐进式披露
 - 6. 显式预加载、Bundle 与 Plugin Skill
 - 7. 自进化总架构：三条闭环
 - 8. Background Review：任务后复盘 Agent 的完整链路
 - 9. skill_manage：六种写动作与写入处理器
 - 10. 安全机制：read-before-write、审批、扫描与回滚
 - 11. 使用遥测、Provenance 与 Curator 生命周期治理
 - 12. LLM Consolidation 与独立 self-evolution 仓库
 - 13. 端到端时序：从用户 Query 到 Skill 更新
 - 14. 工程评价、局限与可研究改进点
 - 15. 本地源码阅读/排查清单
 - 16. 常见追问速答
- 附录 A. 核心源码路径表
- 附录 B. 术语表与配置速查

0. 阅读说明与整合原则

本版不是把两份材料简单拼接，而是按“运行时检索链路 -> 自进化写入链路 -> 安全治理链路 -> 生命周期链路”的工程顺序重新组织。两份材料有交叉：D1 更偏 Skill Set 的表示、索引、查询、缓存和生命周期；D2 更偏 Background Review、skill_manage 写入、安全门与自进化触发。本版将重复内容合并，并把分散在不同章节的关键机制串成可执行的源码级闭环。

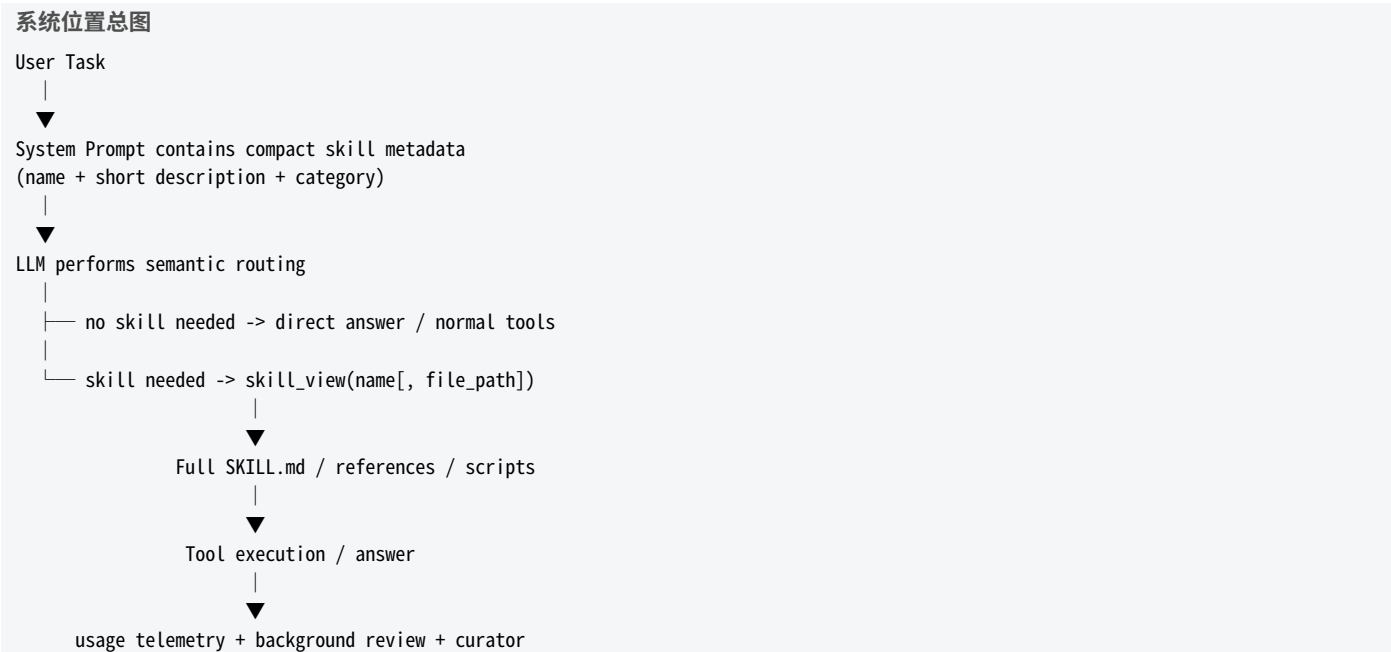
版本边界声明

这份整合严格基于你上传的两份文档。由于没有重新联网校验 Hermes 上游仓库最新提交，文中“当前源码”“近期版本”等表述均指上传材料所分析的版本。若你本地是六月十号下载的源码，某些文件、开关或审批分支不存在，应优先按版本差异处理，而不是认定机制一定缺失。

输入材料	主要贡献	本版如何吸收
D1: Skill Set 源码学习文档	系统性说明 Skill 表示、Index、查询加载、Bundle、Plugin、Curator，以及与 RAG 的区别。	作为“体系结构主线”：先建立 Hermes Skill 的数据模型与运行时加载模型。
D2: Skill 自进化完整源码机制解析	重点解释 Background Review、_skill_nudge_interval、skill_manage 六动作、read-before-write、write approval、guard_agent_created。	作为“自进化主线”：补足触发、写入、安全与审批细节。
两者交叉部分	都强调自进化不是参数训练，而是文件级 procedural memory 的演进。	去重后形成一条完整闭环：经验 -> 文件更新 -> 缓存失效 -> 下次加载 -> 遥测 -> 治理。

1. 一句话总览：Hermes Skill 的系统位置

Hermes Skill 的核心定位是“可执行的程序性记忆层”。它位于模型权重之外、普通对话上下文之外、传统 RAG 文档库之外：以文件系统目录承载可复用工作流，以系统提示中的紧凑目录提供可见性，以工具调用加载完整指令和支持文件，以后台复盘与治理机制持续维护。



这套设计的关键不是“检索到了多少文本 chunk”，而是“在恰当任务上把一套可复用流程、约束、模板和脚本加载进上下文”。因此 Hermes Skill 与传统 RAG 的目标不同：RAG 主要补事实证据，Skill 主要补操作流程。

2. 概念边界：Skill、Memory、RAG、Self-Evolution

概念	Hermes 中的含义	容易误解的点	判断标准
Skill	完成某类任务的可复用程序性知识包，通常是目录 + SKILL.md + 支持文件。	不是单条 prompt，也不是模型参数。	能否指导“怎么做”、调用什么工具、如何验证。
Memory	保存关于用户、环境或世界的声明性事实。	不要把用户偏好、一次性事实塞进 Skill。	是否是“是什么/谁/偏好/历史事实”。
Skill Set	当前运行环境可发现的 Skill 动态集合。	不是常驻数据库对象。	由本地目录、外部目录、插件注册表动态解析。
Skill Index	供 LLM 浏览的 name/description/category 文本目录。	不是向量索引，不一定有 Top-k 分数。	是否进入 system prompt 的紧凑 available skills。
Self-Evolution	有两层：主仓库在线文件级自学习；独立仓库评测驱动的 GEPA/DSPy 优化。	不要把二者混成每轮对话自动强化学习。	看触发、评测数据、是否产生候选竞争。

Memory 与 Skill 的边界例子

“用户使用 Windows”是 Memory，因为它是用户事实；“Windows PowerShell 下不要直接把 ~/.hermes 当 SQLite 路径，需用 \$env:USERPROFILE 展开”是 Skill，因为它是可复用操作规程。

3. Skill 的文件级表示与所有权边界

3.1 一个目录就是一个技能单元

Hermes 的 Skill 以目录为边界。SKILL.md 是入口文件，references/templates/scripts/assets 等目录承载长说明、输出模板、可执行脚本和静态资源。支持文件不会被扫描成独立 Skill，而是由 skill_view(name, file_path=...) 在需要时读取。这样做的目的就是渐进式披露：常驻上下文只付元数据成本，真正需要时才加载正文和长材料。

典型 Skill 目录结构

```
~/.hermes/skills/  
├── research/  
│   ├── arxiv-search/  
│   │   ├── SKILL.md           # 必需: frontmatter + 主流程  
│   │   ├── references/       # 长说明、协议、API 文档  
│   │   │   ├── query-syntax.md  
│   │   │   └── ranking-policy.md  
│   │   ├── templates/       # 报告模板、回复模板  
│   │   │   └── survey-template.md  
│   │   ├── scripts/         # 可执行辅助脚本  
│   │   │   └── search_arxiv.py  
│   │   └── assets/           # 示例数据、图片、JSON 等  
│   │       └── example-query.json
```

3.2 SKILL.md 的两层结构

SKILL.md 通常由 YAML Frontmatter 与 Markdown 正文构成。Frontmatter 用于发现、过滤、配置和路由；Markdown 正文用于实际执行。name 是逻辑标识，description 是最核心的路由信号，platforms/environments/requires/fallback/config 等字段参与过滤或加载时预处理。

SKILL.md 结构示例

```
---  
name: arxiv-search  
description: Search, filter and summarize papers from arXiv  
version: 1.0.0
```

```
platforms: [linux, macos, windows]
metadata:
  hermes:
    category: research
    tags: [paper, arxiv, survey]
    requires_toolsets: [web]
    related_skills: [paper-review]
    config:
      - key: arxiv.max_results
        description: Maximum candidate papers
        default: 20
---
# arXiv Search
## When to Use
## Procedure
## Pitfalls
## Verification
```

3.3 所有权边界：local、external、plugin、bundled、hub

Hermes 没有一个长期驻留的 SkillSet 数据库对象。有效集合来自文件系统与插件注册表。不同来源决定了可见性、优先级和能否被后台自动治理。

来源	解析/优先级	可查看	可被后台自动改写/治理	说明
Local skills: ~/.hermes/skills/	最高；同名时覆盖 external。	是	通常可以，尤其是 agent-created。	用户运行目录，包含 bundled 同步、hub 安装、agent 创建和用户创建内容。
External dirs: config.yaml -> skills.external_dirs	按配置扫描；同名低于本地。	是	自主后台维护应只读；前台明确指令可处理。	外部目录属于外部所有权，避免后台乱改团队共享库。
Plugin skills: namespace:skill-name	通过插件注册表解析。	是	通常不由普通 skill_manage 直接改。	具备 namespace，与本地文件 skill 不同。
Bundled skills: 仓库 skills/ 同步到本地	安装/更新时同步；有 manifest。	是	受保护或需特殊治理规则。	用户本地修改不应被静默覆盖，可 reset。
Hub-installed skills	安装到本地但 provenance 不同。	是	通常排除或受限。	避免后台修改第三方分发内容。

4. Index 构建：扫描、过滤、缓存与上下文注入

4.1 Index 不是向量库

主运行链路中，Skill Index 是结构化文本目录，而不是 embedding/FAISS/Milvus/Qdrant/BM25/reranker。它被渲染进 system prompt，LLM 直接阅读 name、description、category 后做语义路由。没有一个显式的 Top-k 检索结果对象，也没有可观测的 cosine score。

```
系统提示中的紧凑 Index 近似形态

research:
- arxiv-search: Search/filter/summarize arXiv papers...
- paper-review: Critique methods, experiments and claims...

github:
- github-code-review: Review pull requests and code changes...
```

4.2 扫描与过滤流程

1. 确定扫描根：优先扫描本地 ~/.hermes/skills/，然后加入配置中的 external_dirs。

- 2. 递归查找 SKILL.md，同时跳过 .git、虚拟环境、node_modules、缓存目录以及 support directories。
- 3. 解析 Frontmatter，抽取 name、description、platforms、environments、metadata.hermes 等字段。
- 4. 执行 disabled list、平台兼容、环境相关性、requires_toolsets/requires_tools、fallback_for_* 等条件过滤。
- 5. 按 name 去重；本地 skill 先进入 seen set，因此本地同名覆盖外部目录版本。
- 6. 根据相对路径和 DESCRIPTION.md 构造 category 分组。
- 7. 渲染为 system prompt 的 available skills 块，并进入缓存。

4.3 两级缓存：进程内 LRU 与磁盘快照

缓存层	Key/校验信息	作用	局限
进程内 LRU	本地目录、外部目录、可用 tools/toolsets、平台、disabled skills、compact categories 等。	避免同一进程内重复扫描和重复渲染。	进程结束即失效。
磁盘快照 .skills_prompt_snapshot.json	本地 SKILL.md/DESCRIPTION.md 的 mtime + size manifest。	跨进程冷启动复用本地解析结果。	外部目录通常仍直接扫描；快照不是完整正文数据库。
skill_view 读取	按 name/path 读取真实文件。	保证加载的是完整 SKILL.md 或指定支持文件。	不会因为 prompt snapshot 命中而跳过实际正文读取。

Description 的路由风险

D1 特别指出 description 有两种长度语义：skills_list 可保留较长 description，而 system prompt 中的 description 会被压缩到很短的路由片段。因此 description 前几十个字符必须包含最有区分度的任务语义；否则 LLM 可能在路由阶段漏选。

5. 查询与加载：skills_list、skill_view 与渐进式披露

5.1 skills_list 与 skill_view 的职责边界

工具	输入	输出	真正职责	常见误解
skills_list	可选 category。	可用 skill 的 name、description、category 等摘要。	枚举与浏览；辅助调试或当 LLM 不确定时扩大视野。	它不是相似度检索器，不返回 relevance score。
skill_view	name，和可选 file_path。	完整 SKILL.md 或指定支持文件；附 linked files/readiness 等信息。	精确加载被选中的 Skill 内容；进入渐进式披露 Level 1/2。	调用成功后通常同时 bump_view 和 bump_use，不只是“看看”。

5.2 skill_view 的解析与安全检查

- 8. 校验 name，拒绝绝对路径、Windows drive path 和 .. 路径穿越。
- 9. 如果名称包含冒号，按 namespace:skill-name 分发给插件解析。
- 10. 本地/外部 Skill 支持逻辑名、相对类别路径和 Frontmatter name 的解析。
- 11. 当多个候选同名且无法唯一确定时，拒绝猜测，要求更明确路径。
- 12. 读取主 SKILL.md 后执行平台兼容检查、配置注入、linked files 解析、readiness status 计算。
- 13. 读取支持文件时要求 file_path 限制在 Skill 目录内部。
- 14. 成功读取后更新 usage：view/use 计数；在 Background Review 场景还会标记 read-before-write 状态。

5.3 渐进式披露的三层上下文

层级	进入上下文的内容	触发方式	成本/意义
Level 0: 索引	name、短 description、category。	系统提示常驻。	成本随 Skill 数量线性增长；负责 offer surface。
Level 1: 主 Skill	完整 SKILL.md。	LLM 调用 skill_view(name)。	只在被选中时付费；提供完整流程。

Level 2: 支持文件	references/templates/scripts/assets 中的指定文件。	LLM 调用 skill_view(name, file_path)。	只在复杂任务需要时加载；承载长文档、脚本、模板。
---------------	---	-------------------------------------	--------------------------

上下文成本近似模型

Context cost $\approx$ metadata_index
+ I(selected_skill) $\times$ SKILL.md
+ I(needed_support_file) $\times$ support_file

6. 显式预加载、Bundle 与 Plugin Skill

除了自然语言路由，Hermes 还存在显式加载路径。Slash Command 或 CLI 预加载适合用户已经明确知道要使用哪个 Skill 的情况；Bundle 用来一次性加载一组 Skill 与组合指令；Plugin Skill 通过 namespace 解析，管理边界与普通本地目录不同。

机制	作用	是否改变索引本质	典型场景
Slash Command / CLI 预加载	绕过 LLM 是否选择 Skill 的不确定性，直接把指定 Skill 加入上下文并记录 use。	不改变；只是显式指定。	/arxiv-search query、启动时预加载某个工作流。
Bundle	把多个 Skill 与额外组合指令绑定，一次性装载协同工作流。	不是新索引，也不是永久合并目录。	backend-dev 同时加载 TDD、code review、PR workflow。
Plugin Skill	通过 namespace:skill-name 从插件注册表解析，可能带 sibling/bundle context。	与本地文件 skill 解析路径不同。	外部插件提供只读能力或受控能力。

7. 自进化总架构：三条闭环

Hermes Skill 自进化不是“模型自己训练自己”。主仓库内置机制由三条互补闭环组成：任务中使用闭环、任务后复盘写回闭环、长期治理闭环。

三条自进化闭环

Loop A - Runtime use
skills_list / prompt index -> skill_view -> load SKILL.md/support files
-> execute workflow -> bump usage telemetry

Loop B - Post-turn learning
run_conversation ends -> TurnFinalizer decides review_skills
-> spawn_background_review -> fork review_agent
-> whitelist memory/skill tools -> skill_manage create/patch/write_file

Loop C - Long-term governance
usage sidecar -> curator periodic check
-> active/stale/archive deterministic transitions
-> optional LLM consolidation for umbrella skills

闭环	输入信号	核心模块	输出结果	是否更新模型权重
运行时使用	用户任务、系统提示中的 Skill Index、LLM 路由判断。	prompt_builder、skills_tool、skill_usage。	加载 Skill，执行任务，记录 view/use。	否。
任务后复盘	会话快照、工具轨迹、已加载 Skill、用户纠正、非平凡修复经验。	turn_finalizer、background_review、skill_manager_tool。	create/patch/edit/write_file；可能进入审批队列。	否。
长期治理	usage、provenance、state、pin、cron 引用、idle 时间。	skill_usage、curator。	active/stale/archive；可选 consolidation。	否。
独立 self-evolution	评测集、执行轨迹、候选变体、约束门。	hermes-agent-self-evolution，DSPy/GEPA。	评测选择更优 skill/prompt/tool/code，通	仍不是主进程每轮在线训练。

基于上传材料整合，不代表对最新上游 main 的联网复核

			常 PR 合入。	
--	--	--	----------	--

8. Background Review：任务后复盘 Agent 的完整链路

8.1 触发条件：_skill_nudge_interval 与 TurnFinalizer

Background Review 并非每一轮强制触发。D2 中的链路是：conversation_loop 在工具调用迭代中维护 _iters_since_skill；若 skill_manage 可用且 _skill_nudge_interval > 0，迭代计数会增加。任务结束时，turn_finalizer 检查计数是否达到阈值、有无最终回复、是否中断，以及是否需要 memory/skill review。满足条件后调用 _spawn_background_review。

```
触发逻辑的概念化伪代码

_should_review_skills = (
    agent._skill_nudge_interval > 0
    and agent._iters_since_skill >= agent._skill_nudge_interval
    and "skill_manage" in agent.valid_tool_names
)

if final_response and not interrupted and (review_memory or review_skills):
    agent._spawn_background_review(
        messages_snapshot=list(messages),
        review_memory=_should_review_memory,
        review_skills=_should_review_skills,
    )
```

_skill_nudge_interval 的含义

它不是“每隔多少秒学习一次”，而是“工具调用迭代达到一定间隔后，提醒/允许任务后 Skill Review”。作用是避免每轮对话都启动后台复盘，同时保证复杂工具任务后有机会沉淀经验。

8.2 后台 review_agent 的运行隔离

Background Review 是响应交付之后运行的后台审稿 Agent。它拿到当前会话快照，fork 一个新的 AI Agent，继承 provider/model/platform 等运行配置，但关闭真实会话持久化，并用工具白名单限制它只能使用 memory/skill management 相关工具。它不负责继续回答用户，也不应调用普通任务工具。

隔离项	典型设置/行为	目的
quiet_mode	True	后台运行，不干扰用户主任务输出。
max_iterations	有限迭代，例如 16	控制后台成本，防止无限复盘。
skip_memory / _persist_disabled	跳过真实会话持久化，不写 session DB/json。	避免 review 对话污染真实会话记录。
tool whitelist	只允许 skills 和可选 memory 工具。	防止后台 Agent 继续执行用户任务或调用不相关工具。
messages_snapshot	复制主任务对话与工具轨迹。	给 review_agent 足够证据判断是否值得沉淀。

8.3 _SKILL_REVIEW_PROMPT 的决策边界

Background Review 的提示词倾向于主动发现可复用经验，但不是无条件保存。它强调 class-level skills：应该沉淀某一类任务的通用流程，而不是“一轮会话一个 Skill”。优先级是：先更新本轮真正加载/使用过的 Skill；其次更新已有 umbrella Skill；再考虑把长材料下沉到 references/templates/scripts；最后才创建新的类别级 Skill。

应当保存的信号	不应保存的内容	原因
用户纠正了风格、工作流或方法。	一次性任务叙事。	Skill 要服务未来同类任务，不是复述历史。
任务中出现非平凡技术、修复、绕过方案、debug pattern、tool pattern。	环境依赖的临时故障。	临时故障不一定可迁移，容易污染流程。

已加载 Skill 被证明错误、缺步骤或过时。	对工具“坏了”的负面结论。	工具异常可能是 transient，不应固化。
可抽象为 fix/procedure/reusable workflow 的经验。	没有验证的一次成功路径。	要保存可复用机制，而不是偶然结果。

Class-first 的实际含义

“class-level”意味着 Skill 要覆盖一类任务，例如“Python 包安装冲突排查”或“论文批量阅读总结”，而不是“今天帮用户修某个具体报错”。这能控制 Skill 膨胀，也便于未来路由。

9. skill_manage：六种写动作与写入处理器

Background Review 本身不直接写文件，它只能调用 skill_manage。skill_manage 是文件级 procedural memory 的写入口，支持 create、patch、edit、delete、write_file、remove_file 六类动作。

Action	源码语义	适用场景	工程注意点
create	创建新 Skill 目录并写入 SKILL.md。	没有合适 Skill，且经验具有类别级复用价值。	校验 name/category/frontmatter/content size；检查重名；原子写入；安全扫描失败可 rollback。
patch	用 old_string/new_string 精确替换，可支持 fuzzy matching/replace_all。	修复已有 Skill 中的小错误、更新命令、补充一个步骤。	首选方式；old_string 应唯一；patch SKILL.md 后要重新校验 frontmatter。
edit	整体替换 SKILL.md。	结构性重写、大规模重构。	风险高，不适合小改；后台通常要先读。
delete	删除或归档 Skill。	废弃、合并、absorbed_into 到 umbrella。	自动治理路径通常偏 archive，不轻易永久删除。
write_file	新增/覆盖 references/templates/scripts/assets 中的支持文件。	把长材料、模板、脚本从 SKILL.md 下沉。	检查路径合法、大小限制、目标存在；若覆盖已有文件，后台必须先读过该文件。
remove_file	删除支持文件。	清理过时 reference/template/script。	同样受路径、安全和 read-before-write 约束。

9.1 create 的处理链

```
create 写入链
_create_skill:
1. validate name / category / frontmatter
2. enforce SKILL.md size limit
3. check duplicate skill name
4. create target directory
5. atomic write SKILL.md
6. optional guard_agent_created security scan
7. rollback if dangerous finding blocks write
8. clear prompt cache + mark provenance/usage
```

9.2 patch 的处理链

```
patch 写入链
_patch_skill:
1. resolve target skill and optional file_path
2. read current content
3. require old_string unique unless replace_all/fuzzy path is used
4. produce new content by replacement
5. if patching SKILL.md, validate frontmatter again
```

- 6. atomic write
- 7. optional security scan
- 8. rollback on failure
- 9. bump patch telemetry and clear skill prompt cache

9.3 write_file 的处理链

write_file 适合把长参考资料、模板或脚本拆到支持目录。它不是降低专业性，而是提升 Skill 的可维护性：SKILL.md 保持短流程，references/scripts 承载复杂细节。

支持文件示例

```
write_file examples:
- references/debugging-checklist.md
- templates/paper-summary-template.md
- scripts/collect_logs.py
- assets/example-query.json
```

10. 安全机制：read-before-write、审批、扫描与回滚

自进化最容易出问题的地方不是“能不能写”，而是“是否凭空写、是否写错对象、是否覆盖用户资产、是否把危险脚本写进去”。Hermes 在 D2 所述版本中包含多层安全门。

10.1 read-before-write：防止后台凭印象改文件

Background Review 想 patch/edit 已有 Skill，必须先通过 skill_view 读取目标文件。skill_view 会记录后台 review 已读路径，skill_manage 写入时检查该标记。若要改支持文件，也必须先 skill_view(name, file_path= "...") 读取对应文件。

read-before-write 守卫

```
Background Review wants to patch SKILL.md
|
├─ must call skill_view(name)
|   └─ mark_background_review_skill_read(name, SKILL.md)
|
└─ then skill_manage(action="patch"/"edit")
    └─ _background_review_read_before_write_guard verifies read mark
```

10.2 Background Write Guard：防止后台越权改资产

后台 review/curator 会拒绝修改某些目标，例如 pinned skills、external skill paths、protected built-ins、hub-installed skills、bundled skills。核心思想是：后台自动学习只能安全地维护自己所有权的内容；用户显式前台指令可以有更高权限，但后台不能擅自改共享或受保护内容。

被保护对象	为什么保护	后果
Pinned skills	用户明确认为重要或不希望自动治理。	跳过自动迁移/改写。
External skill paths	外部目录可能属于团队或其他项目。	后台自主维护应只读。
Protected built-ins	系统承重技能被改坏会影响核心能力。	拒绝后台写入或归档。
Hub-installed / bundled skills	第三方/分发内容不应被静默改写。	通常只允许用户明确操作或 reset。

10.3 write approval: 人审闸门

如果开启 `skills.write_approval=true`, `skill_manage` 的 `create/edit/patch/delete/write_file/remove_file` 不会立即落盘, 而是进入 `pending queue`。用户可以 `review diff`、`approve` 或 `reject`。`approve` 后由 `apply_skill_pending` 在 `bypass gate` 情况下重放写操作。

写审批两种模式

Default mode:

Background Review -> skill_manage -> validate -> atomic write -> clear cache

Approval mode:

- Background Review -> skill_manage -> pending queue
 - > user reviews diff
 - > approve/reject
 - > apply pending write if approved

10.4 guard_agent_created: 机器安全扫描

guard_agent_created 是可选 security scanner，默认关闭。启用后，它扫描 Agent 创建的 Skill 或支持文件；如果发现 dangerous findings，可以阻断或回滚。它与 write approval 的区别是：write approval 是人类审批门，guard_agent_created 是机器安全扫描门。二者可以同时开启。

10.5 原子写、校验与缓存失效

写入不是简单覆盖字符串。处理器会校验 name/frontmatter/路径/大小，patch 时要求 old_string 唯一或走 fuzzy path，写入采用原子替换，并在成功后清除 skills system prompt cache。这样下一次构建 prompt 时，文件 manifest 变化会导致索引重新扫描。

11. 使用遥测、Provenance 与 Curator 生命周期治理

11.1 .usage.json: 为什么不写进 Frontmatter

使用统计存放在 `~/hermes/skills/.usage.json` sidecar，而不是写进 `SKILL.md` Frontmatter。这样可以避免运行时遥测污染可共享内容，也减少 `bundled/hub Skill` 合并冲突。计数写入通常使用文件锁和原子替换，失败只记录 `DEBUG`，不影响正常任务。

.usage.json 概念示例

```
{
  "arxiv-search": {
    "created_by": "agent",
    "view_count": 12,
    "use_count": 9,
    "patch_count": 3,
    "last_viewed_at": "...",
    "last_used_at": "...",
    "last_patched_at": "...",
    "state": "active",
    "pinned": false
  }
}
```

11.2 Provenance: 谁创建决定谁能被自治治理

D2 强调只有 `action==create` 且处于 `background_review()` 时，才标记为 `agent-created`。前台用户创建的 Skill 更接近 `user-owned`，不应默认被 Curator 当作自治对象随意管理。这种 `foreground/background` 区分解释了为什么同样在 `~/hermes/skills/` 下，不同 Skill 的治理策略不同。

来源/状态	自治治理倾向	原因
agent-created	标准 Curator 候选。	由系统自动沉淀，系统也应负责清理、合并和归档。
foreground user-created	默认更谨慎。	用户资产，不应因低频使用就被自动处理。
bundled/hub/external	默认受限或排除。	分发/外部所有权边界。
pinned	跳过自动迁移。	用户显式保护。

11.3 Curator 确定性生命周期

Curator 负责长期治理。确定性迁移不依赖 LLM，主要根据最近活动时间、provenance、pin、cron 引用、protected built-ins 等条件执行 active -> stale -> archived。归档通常移动到 .archive/，不是永久删除，因此可恢复。

参数	默认值/材料中说明	含义
interval_hours	168 小时 (7 天)	距离上次维护超过该周期才考虑运行。
min_idle_hours	2 小时	Agent 空闲达到阈值后才触发后台维护。
stale_after_days	30 天	长期无活动 active Skill 标为 stale。
archive_after_days	90 天	更长期无活动 Skill 移入 .archive/。
consolidate	默认 false	LLM umbrella-building 合并默认关闭。

Curator 生命周期状态机

```

active
  | no activity >= stale_after_days
  ▼
stale
  | no activity >= archive_after_days
  ▼
archived -> ~/.hermes/skills/.archive/ -> restorable
  
```

不能简单说 Curator 只处理 agent-created

D1 指出某些版本中 Curator 还提供 prune_builtins 配置，可让 bundled built-ins 进入确定性归档候选，但少数 load-bearing Skill 有硬保护，Hub Skill 仍被排除。因此阅读旧材料或本地旧版本时，需要区分版本。

12. LLM Consolidation 与独立 self-evolution 仓库

12.1 Curator 的可选 LLM consolidation

consolidation 是 Curator 的可选主观维护阶段。当 curator.consolidate=true 或显式运行 --consolidate 时，辅助模型可以审查重叠技能，执行 umbrella-building、patch、archive 等。它默认关闭，因为它有模型成本，而且合并技能会改变语义边界，风险高于确定性 stale/archive。

umbrella skill 合并示意

```

docker-build-fix
docker-network-fix
docker-volume-fix
docker-compose-debug
    └─> docker-troubleshooting
        └─ SKILL.md
            └─ references/*.md
  
```

12.2 主仓库在线自进化 vs 独立 self-evolution

维度	主仓库在线自进化	独立 hermes-agent-self-evolution
触发	任务后 Background Review 或周期 Curator。	显式优化运行。
输入	会话经验、工具轨迹、已加载 Skill、usage。	Eval Dataset、Execution Trace、Constraint

基于上传材料整合，不代表对最新上游 main 的联网复核

		Gates。
候选竞争	较弱；多为直接 patch/create。	生成多个候选变体，跑评测后选择。
优化对象	SKILL.md、支持文件、生命周期状态。	Skills、tool descriptions、system prompts、code 等。
质量保证	read-before-write、审批、扫描、归档保护。	tests、size limit、semantic preservation、PR review 等。
与强化学习关系	不是在线 RL，不更新模型参数。	更像评测驱动的程序/提示词演化搜索，也不等同于模型参数强化学习。

为什么别人会把自进化和强化学习联系起来

外部 self-evolution 仓库有“生成候选 -> 评测 -> 选择更优版本”的优化味道，容易被泛化成“强化学习式反馈优化”。但从上传材料看，Hermes 主仓库的日常自进化没有参数更新、没有策略梯度、没有 reward model 训练；它是程序性知识文件的写回与治理。

13. 端到端时序：从用户 Query 到 Skill 更新

下面给出一个完整例子：用户要求检索并总结 Agent Skill Retrieval 论文。该例同时覆盖运行时路由、加载、执行、写回和治理。

- 15. 用户输入自然语言任务。
- 16. System Prompt 中已经包含 research 类别的短索引。
- 17. LLM 根据 description 判断 arxiv-search / paper-review 等 Skill 是否相关；如果不确定，可能先调用 skills_list。
- 18. LLM 调用 skill_view("arxiv-search")，完整 SKILL.md 进入上下文；工具成功后 bump_view + bump_use。
- 19. 若主文档引用 query syntax 或 ranking policy，LLM 再调用 skill_view(name, file_path="references/...")。
- 20. Agent 按 Skill 流程调用 web/search 等工具，完成筛选、核验、总结。
- 21. 如果任务中发现 arXiv 时间过滤步骤有 bug，前台 Agent 或 Background Review 先 skill_view 读取原 Skill。
- 22. skill_manage(action="patch") 用精确 old_string/new_string 修正步骤；若开启审批则进入 pending。
- 23. 写入成功后清除 prompt cache；文件 mtime/size 变化使 snapshot manifest 下次失效。
- 24. usage sidecar 更新 last_used/last_patched；Curator 因近期活动保持 active，或长期无活动时迁移到 stale/archive。

端到端时序图

```
User -> LLM -> Prompt Index
      -> skill_view("arxiv-search")
      -> Skill Files: SKILL.md + references/query-syntax.md
      -> Execute search/summarization workflow
      -> Answer
      -> Background Review
          -> skill_view(read target)
          -> skill_manage(patch/write_file)
          -> clear cache + bump usage
      -> Curator later observes usage and state
```

14. 工程评价、局限与可研究改进点

14.1 工程优点

- 可解释：Skill 是 Markdown、脚本和模板，所有规则与修改可审计。
- 可移植：目录可通过 Git、Hub、URL、external_dirs 或插件共享。
- 可执行：不是单纯提示词，能携带 scripts/templates/assets。
- 上下文高效：索引常驻，正文和支持文件按需加载。
- 持续学习：任务经验可以转化为可复用 procedural memory。
- 生命周期治理：usage + provenance + Curator 可控制技能膨胀。

14.2 核心局限

局限	原因	实际后果	可能改进
Need Awareness 不稳定	是否需要 Skill 由 LLM 自行判断。	模型可能直接回答，错过本可提升质量的 Skill。	加入显式 need detector 或策略：复杂工具任务默认浏览相关类别。
Relevance Awareness 依赖 description	没有独立 Retriever，短描述承载主要路由。	描述差会漏检；同义技能会竞争。	description 规范化 + hybrid retrieval + LLM rerank。
规模扩展问题	System Prompt 索引成本近似 O(N)。	Skill 很多时 token 与注意力成本上升。	分层目录、按域动态 offer、向量/关键词候选裁剪。
无显式检索分数	选择发生在 LLM 内部。	难做 Recall@K、归因和路由评估。	记录候选、选择理由和反事实实验。
在线写回可能污染	一次成功不一定代表通用流程。	固化偶然路径或环境特定经验。	写入前抽象化、审批、回归任务集验证。
Curator 评价有限	确定性治理看活动，LLM 合并默认无任务评测。	低频重要技能可能被误判，合并质量不稳定。	引入任务覆盖率、失败率、回归测试和重要性标注。

14.3 面向研究的强化方案

可研究的 Skill Retrieval + Evolution Gate 框架

Candidate generation

metadata / keyword retrieval

embedding retrieval

task-tool compatibility filter

LLM need + relevance judge

▼

Hierarchical loading

▼

Execution trace + success/cost/safety metrics

▼

Patch candidate generation

▼

Regression evaluation against old skill

▼

Promote only if statistically or operationally better

- Need Awareness：先判断任务是否需要外部程序性知识，而不是直接全库检索。
- Relevance Awareness：采用 BM25/embedding/metadata 的 hybrid retrieval 产生候选，再由 LLM 给出选择理由。
- Executable Validation：不仅测文本相关性，还要测工具可用性、脚本成功率、前置条件满足率。
- Evolution Gate：patch/create 后必须在历史任务集上做回归，显著优于旧版本再晋升。
- Skill Graph：显式维护 related_skills、fallback、替代关系、前置依赖和 Bundle，而不是只靠 description。

15. 本地源码阅读/排查清单

如果你本地源码找不到某个分支、开关或文件，首先按版本差异排查。六月十号下载的源码可能早于上传材料分析的 main 分支。下面是建议的源码阅读顺序。

顺序	文件/模块	重点看什么	验证问题
1	tools/skills_tool.py	skills_list、skill_view、路径安全、linked files、usage bump、mark_background_review_skill_read。	Skill 如何被发现与加载？支持文件如何读取？
2	agent/skill_utils.py	Frontmatter 解析、目录来源、平台/环境/工具条件过滤、description truncation。	为什么某些 Skill 没出现在可用列表？
3	agent/prompt_builder.py	build_skills_system_prompt、LRU cache、snapshot manifest、category rendering。	Index 如何进入 System Prompt？缓存何时失效？
4	agent/conversation_loop.py	工具调用迭代、_iters_since_skill 计数。	任务中如何积累 review 信号？
5	agent/turn_finalizer.py	_should_review_skills、_spawn_background_review。	任务结束后何时启动后台复盘？
6	agent/background_review.py	review_agent fork、tool whitelist、	后台复盘到底运行什么？

基于上传材料整合，不代表对最新上游 main 的联网复核

		_SKILL_REVIEW_PROMPT。	
7	tools/skill_manager_tool.py	create/edit/patch/delete/write_file/ remove_file、approval gate、read- before-write。	后台如何写文件？为何被拒绝？
8	tools/skill_usage.py	usage sidecar、atomic write、view/use/patch、state、pin。	遥测如何记录？Curator 凭什么判断活跃？
9	agent/curator.py	idle trigger、active/stale/archive、 consolidate、protected skills。	生命周期治理与合并如何发生？
10	agent/skill_commands.py	Slash command、Bundle、preload、 activation note。	显式加载和自然语言路由有什么区别？

本地没有 approve 分支怎么办

先搜索 write_approval、pending、apply_skill_pending、_apply_skill_write_gate 等关键字。如果都没有，大概率是你本地版本早于该机制，或分支/配置名不同。不要只在 UI 层找 approve；审批可能在 skill_manager_tool 或 CLI/管理命令路径中。

16. 常见追问速答

问题	答案
Q1：Hermes Skill 自进化是不是强化学习？	主仓库不是。它不更新模型参数，不做策略梯度，也没有 reward model 训练。它是文件级 procedural memory 写回、审批、遥测和治理。独立 self-evolution 仓库有候选生成与评测选择，但也应理解为评测驱动的程序/提示词演化，而非主进程在线 RL。
Q2：为什么看起来没有向量检索？	因为主链路就是轻量元数据目录进 System Prompt，LLM 根据 name/description/category 做语义路由，再用 skill_view 精确加载。没有显式 Top-k、embedding 或 reranker。
Q3：skills_list 是检索器吗？	不是。它枚举可用 Skill 摘要，便于浏览和调试；真正“选不选”仍由 LLM 判断。
Q4：skill_view 只是浏览吗？	在材料所述工具注册包装器中，成功 skill_view 会同时 bump_view 和 bump_use，因为 Agent 主动加载完整 Skill 通常意味着要用于当前任务。
Q5：telemetry 是什么？	就是使用遥测：view/use/patch 次数、last_used_at、state、pinned、created_by 等运行数据，写在 .usage.json sidecar 中，供 Curator 做生命周期判断。
Q6：umbrella Skill 是什么？	把多个重复的微技能合并到一个更高层的类别级技能中。例如 docker-build-fix、docker-network-fix 合并为 docker-troubleshooting，细节下沉到 references。
Q7：consolidation 是什么？	Curator 的可选 LLM 合并/整理阶段。默认关闭。它会审查重叠 Skill，尝试 umbrella-building、patch、archive 等主观维护。
Q8：为什么后台会拒绝修改某些 Skill？	因为有所有权边界和安全门：pinned、external、protected built-ins、hub-installed、bundled 等不应被后台自动改写。
Q9：本地源码找不到 curator.consolidate 开关怎么办？	按版本差异处理。六月十号下载的版本可能早于材料分析版本；搜索 consolidate、curator、umbrella、archive、stale 等关键字，并检查 release notes 或 main 分支差异。
Q10：Description 怎么写才利于路由？	前 60 个字符必须包含任务类型、触发条件和边界。不要以泛化口号开头；例如 “Debug Python package import/path issues on Windows/conda” 比 “Helps with Python” 更可路由。

附录 A. 核心源码路径表

源码路径	角色	你应该重点验证的机制
agent/skill_utils.py	Skill 元数据与条件过滤工具。	Frontmatter parsing、get_all_skills_dirs、iter_skill_index_files、requires/fallback、description 截断。
agent/prompt_builder.py	构造 system prompt 中的 available skills。	build_skills_system_prompt、LRU cache、snapshot manifest、compact categories。
tools/skills_tool.py	运行时查看与加载 Skill。	skills_list、skill_view、路径安全、plugin namespace、linked files、readiness、bump usage。

Hermes Agent Skill Set 机制整合 - 详细专业版

agent/skill_commands.py	Slash command、Bundle、preload。	显式加载如何生成 activation note、如何 bump_use。
tools/skill_manager_tool.py	Skill 创建/编辑/补丁/删除入口。	action schema、write approval、read-before-write、background write guard、atomic write、rollback。
tools/skill_usage.py	遥测 sidecar。	.usage.json、view/use/patch counts、last_activity、state、pin、archive/restore。
agent/background_review.py	任务后复盘。	spawn_background_review、review_agent fork、tool whitelist、_SKILL_REVIEW_PROMPT。
agent/turn_finalizer.py	回合结束触发 review。	_should_review_skills、messages_snapshot、是否中断/是否最终回复。
agent/conversation_loop.py	任务中迭代计数。	_iters_since_skill、_skill_nudge_interval。
agent/curator.py	生命周期治理。	idle trigger、active/stale/archive、pinned/protected、LLM consolidation。
hermes-agent-self-evolution 独立仓库	评测驱动的外部演化器。	DSPy/GEPa、候选变体、测试/约束门、PR 合入。

附录 B. 术语表与配置速查

术语/配置	解释
procedural memory	程序性记忆，即“如何做”的任务流程、工具调用规范、验证步骤。Hermes Skill 属于这一层。
declarative memory	声明性记忆，即“是什么”的事实、偏好、环境信息。
progressive disclosure	渐进式披露：先展示短索引，按需加载 SKILL.md，再按需加载支持文件。
offer surface	模型当前可见的能力表面，即 system prompt 中出现的 Skill 元数据目录。
read-before-write	后台改文件前必须先 skill_view 读取目标，防止凭会话印象盲改。
write_approval	写审批门；开启后写入进入 pending，用户 diff/approve/reject 后才落盘。
guard_agent_created	可选机器安全扫描；发现危险内容时阻断或回滚 Agent 创建的 Skill。
usage sidecar	~/hermes/skills/.usage.json，用于存放 view/use/patch/state/pinned/provenance 等运行遥测。
provenance	来源/所有权信息；区分 agent-created、foreground user-created、bundled、hub、external 等。
stale/archive	Curator 生命周期状态；长期无活动先 stale，再 archive，可恢复。
umbrella skill	用于合并多个重复微技能的上层技能，细节可下沉到 references。
consolidation	Curator 的可选 LLM 主观整理/合并阶段，默认关闭。
_skill_nudge_interval	工具调用迭代达到一定间隔后触发 Skill Review 判断的阈值，不是时间间隔。
compact_categories	某些运行姿态下把类别降级为只显示名称、不显示描述，以压缩 system prompt。

结语：最应该记住的模型

最终心智模型

Hermes Skill = 文件夹式程序性记忆

运行时：
文件系统扫描 -> Frontmatter 元数据 -> System Prompt 索引
-> LLM 路由 -> skill_view 精确加载 -> 工具执行

自进化：
任务经验/用户纠正 -> Background Review -> skill_manage 写文件
-> 安全门/审批/扫描 -> 缓存失效 -> 下次重新索引
-> usage telemetry -> Curator stale/archive/consolidate

不是：
不是普通 prompt 字符串；不是向量 RAG；不是在线训练模型参数。

补充专题：源码级精读、版本排查与研究化扩展

前面的章节已经给出完整机制链路。下面补充更偏“源码阅读”和“工程复现”的内容，用于解决两个实际问题：第一，本地源码和文档描述不一致时如何定位；第二，如果要把 Hermes Skill 机制发展成研究或项目，应该如何做实验和设计门控。

17. 源码模块逐文件精读指南

阅读 Hermes Skill 子系统时，不建议从仓库根目录泛读，而应按“可见性 -> 加载 -> 使用 -> 写入 -> 治理”的顺序追踪。每个文件要回答一个明确问题，否则容易把 Skill Index、skill_view、Background Review 和 Curator 混在一起。

模块	首先回答的问题	关键断点/搜索词	读完后应能解释
agent/skill_utils.py	哪些目录会被扫描？哪些 Skill 会被过滤？	parse_frontmatter, platforms, environments, requires_toolsets, fallback_for, disabled	为什么某个 Skill 出现在/不出现在 system prompt 中。
agent/prompt_builder.py	元数据如何被压缩并注入系统提示？	build_skills_system_prompt, snapshot, manifest, compact_categories	为什么修改 SKILL.md 后需要缓存失效；为什么外部目录可能不进快照。
tools/skills_tool.py	模型怎样从摘要加载完整 Skill？	skills_list, skill_view, file_path, path traversal, bump_view, bump_use	skills_list 不是检索器；skill_view 是精确读取器并带安全约束。
agent/conversation_loop.py	工具调用如何累计学习信号？	_iters_since_skill, _skill_nudge_interval	为什么不是每轮都学习，而是按工具迭代节流。
agent/turn_finalizer.py	回合结束后何时启动 review？	_should_review_skills, final_response, interrupted, _spawn_background_review	Background Review 为什么在响应交付后运行。
agent/background_review.py	后台 review_agent 如何被隔离？	quiet_mode, max_iterations, whitelist, _SKILL_REVIEW_PROMPT	后台只沉淀经验，不继续做用户任务。
tools/skill_manager_tool.py	写入是如何被校验、审批和回滚的？	skill_manage, write_approval, read_before_write, write_guard, atomic write	为什么后台改某些 Skill 会被拒绝；patch 为什么优先于 edit。
tools/skill_usage.py	遥测如何变成治理依据？	.usage.json, bump_use, bump_patch, last_activity, pinned, state	Curator 如何判断 active/stale/archive。
agent/curator.py	长期维护何时发生？怎么合并和归档？	interval_hours, min_idle_hours, stale_after_days, archive_after_days, consolidate	确定性生命周期与可选 LLM consolidation 的区别。

18. Background Review 到落盘的逐步跟踪

如果你要在源码里真正跟一遍自进化，需要把“触发、构造后台 Agent、决策、读取、写入、安全门、缓存失效”串起来。下面按一次 patch 更新已有 Skill 的场景拆解。

```
patch 更新已有 Skill 的完整链路

Step 0: Main task uses tools several times
conversation_loop increments _iters_since_skill

Step 1: Turn finalization
turn_finalizer checks final response / interruption / review flags
-> _spawn_background_review(messages_snapshot, review_skills=True)

Step 2: Background thread
background_review creates review_agent
-> quiet_mode=True
-> persistence disabled
-> tool whitelist = skill/memory tools
-> injects _SKILL_REVIEW_PROMPT

Step 3: Decision
review_agent inspects conversation + tool traces + loaded skills
-> decides whether the signal is reusable procedure or one-off fact
```

```
Step 4: Read before write
review_agent calls skill_view(target)
-> marks target path as read for background review

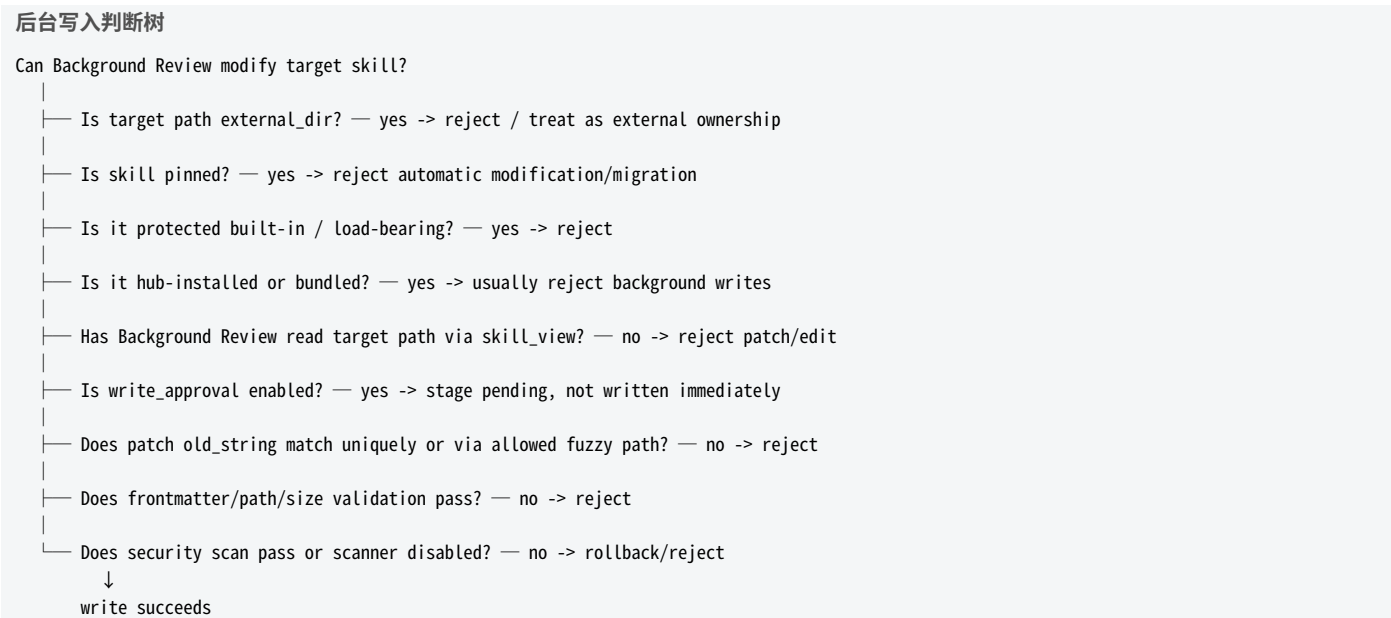
Step 5: Write
review_agent calls skill_manage(action="patch", old_string=..., new_string=...)
-> background write guard checks ownership
-> approval gate may stage pending write
-> handler validates content and atomically writes
-> clears skill prompt cache and bumps usage/provenance
```

定位失败点的方法

如果 patch 没有发生，不要只看最终文件是否变化。按顺序检查：是否达到 `_skill_nudge_interval`；`turn_finalizer` 是否触发 `review_skills`；`background review` 是否启动；`skill_manage` 是否在 `whitelist` 中；是否先 `skill_view` 读取目标；是否被 `background write guard` 拒绝；是否开启 `write_approval` 导致进入 `pending`；是否 `security scanner` 回滚。

19. 写入被拒绝的判断树

后台拒绝修改某些 Skill 并不是 bug，而是所有权和安全边界。下面给出一个判断树，可用于解释“为什么后台拒绝修改某个 skill”。



拒绝原因	表面现象	应该怎么查
未 read-before-write	skill_manage 返回要求先读或无权限修改。	看 review_agent 是否调用 skill_view(name) 或 skill_view(name,file_path)。
目标属于 external/bundled/hub/protected	后台写入被拒绝，但前台用户显式操作可能允许。	检查 provenance、manifest、external_dirs、protected list。
开启 write_approval	文件没变，但存在 pending 记录。	搜索 pending queue、apply_skill_pending、review/approve/reject 命令。
patch old_string 不唯一或不匹配	patch 失败或要求更精确字符串。	打印目标文件片段，缩小 old_string；避免 edit 大重写。
frontmatter/size/path 校验失败	创建或写支持文件失败。	检查 name 规则、SKILL.md 大小、支持文件路径是否越界。
security scan 回滚	短暂写入后恢复或返回 dangerous finding。	看 guard_agent_created 配置和 scanner 日志。

20. 本地版本差异排查：为什么你电脑上找不到这些机制

你之前提到“六月十号下载的源码”“本地没有 curator consolidation 开关”“没找到 approve 分支”。这类情况通常来自版本差异、分支差异、配置默认值差异或文件路径重构。专业做法是把机制拆成关键字组逐个检索，而不是只找一个函数名。

要验证的机制	优先搜索关键词	如果没有结果，可能说明
写审批 gate	write_approval, pending, apply_skill_pending, approve, reject, review diff, _apply_skill_write_gate	本地版本早于审批机制；或审批入口在 CLI/管理命令而非 skill_manager_tool。
后台 read-before-write	mark_background_review_skill_read, read_before_write, background_review_read, skill_view read mark	本地版本还没有该安全门，后台可能直接写或完全不支持后台写。
Curator consolidation	consolidate, umbrella, curator.consolidate, --consolidate, absorbed_into	本地版本未加入 LLM consolidation，只有 deterministic stale/archive。
Built-in pruning	prune_builtins, protected builtins, load-bearing, bundled	本地 Curator 可能只处理 agent-created，尚未扩展到 built-ins。
Skill nudge	_skill_nudge_interval, _iters_since_skill, review_skills	本地触发逻辑可能在 turn finalizer 或 conversation loop 之外，或命名不同。
Usage telemetry	.usage.json, bump_view, bump_use, bump_patch, last_activity	如果没有 sidecar，可能只有简单使用计数或尚未引入 Curator。

推荐的版本确认命令

在仓库根目录运行：git rev-parse HEAD；git branch --show-current；git log -n 5 --oneline；git grep -n "write_approval\|apply_skill_pending\|consolidate\|_skill_nudge_interval\|mark_background_review_skill_read"。把结果和文档中的源码路径逐项对齐。

21. 如何判断应该 create、patch、write_file、consolidate 还是 archive

自进化不是“有新经验就写”。专业系统应当用一套决策准则决定写入动作，否则 Skill 库会迅速膨胀并污染路由。

场景	推荐动作	理由	反例
已有 Skill 的一个步骤错了或命令过时	patch	最小修改，保留历史结构，风险低。	不要 edit 整个 SKILL.md。
已有 Skill 缺少一段长参考说明	write_file 到 references/	主文档保持短流程，长内容渐进加载。	不要把大段 API 文档塞进 SKILL.md。
已有 Skill 缺少输出格式模板	write_file 到 templates/	模板可复用，且按需加载。	不要在多个 Skill 复制同一模板。
需要可执行辅助逻辑	write_file 到 scripts/	把确定性处理交给脚本，降低 LLM 随机性。	不要在 prompt 中要求模型手算复杂转换。
完全没有同类 Skill，且经验可复用	create	新增类别级能力。	不要为一次性任务创建 Skill。
多个微技能高度重复	consolidate/umbrella + archive absorbed skills	减少路由竞争和维护成本。	不要把不相关任务硬合并。
长期不用、非 pinned、非保护对象	stale/archive	降低 offer surface 和维护负担。	不要永久删除低频但关键技能。

21.1 写入质量 Rubric

- 通用性：新内容是否适用于一类任务，而非仅适用于本轮会话？
- 可验证性：是否包含验证步骤、失败检查或安全边界？
- 最小性：能 patch 就不 edit；能写 reference/template/script 就不膨胀 SKILL.md。
- 可路由性：description 前部是否明确说明任务类型和触发条件？
- 可维护性：是否避免复制粘贴长文档；是否把资源放进合适子目录？
- 安全性：脚本是否可能执行危险命令；是否需要审批或扫描？
- 回归风险：修改后是否会破坏已有任务流程？是否需要历史任务集测试？

22. Hermes Skill 与 JSON Skill 库/传统 Prompt 库的差异

你之前也问过 SkillIX、JSON skill library、Claude Code 文件夹式 skill。Hermes 的设计更接近 Claude Code/Anthropic-style skill folder，而不是 SkillIX 那种单条 JSON skill。差异主要体现在资源承载能力、渐进式披露和可执行性。

维度	Hermes 文件夹式 Skill	JSON/条目式 Skill	传统 Prompt 库
基本单元	目录 + SKILL.md + references/templates/scripts/assets。	一条结构化 JSON，通常含 name/description/instruction。	一段文本模板。
可携带资源	强；可附脚本、模板、示例、静态资源。	弱；通常需要外部字段或 URL。	弱；多为纯文本。
渐进式披露	天然支持：元数据 -> 主文档 -> 支持文件。	取决于系统实现。	通常全部塞入或人工选择。
执行能力	可把确定性流程交给 scripts。	通常不直接携带脚本。	没有资源边界。
版本治理	可用 Git/目录/provenance/usage/curator。	依赖数据库版本字段。	常靠人工维护。
路由信号	description + category + related skills。	JSON metadata。	文件名或人工约定。
缺点	目录扫描和 prompt index 随规模膨胀；路由可观测性弱。	资源能力弱；复杂技能容易退化成长文本。	难治理、难验证、难携带工具链。

23. 如果把 Hermes Skill 机制做成研究，实验怎么设计

Hermes 当前强在工程机制，弱在可量化检索与演化评估。如果要研究 Agent Skill Retrieval/Evolution，可以把 Hermes 表示层作为 baseline，再引入检索、门控和回归评测。

23.1 推荐实验任务

- Tool-use tasks：需要多步工具调用、文件读写、网页检索或代码执行的任务。
- Debug tasks：存在非平凡修复路径，适合检验 Background Review 能否沉淀 debug pattern。
- Document generation tasks：需要模板、格式约束、验证步骤，适合检验 templates/references 的价值。
- Coding tasks：需要 scripts/CI/test，适合比较“纯 prompt skill”和“带脚本 skill”。
- Longitudinal tasks：多轮会话后 Skill 库膨胀，适合测 Curator 与 consolidation。

23.2 评价指标

指标	衡量什么	如何采集
Need Recall	本该使用 Skill 的任务中，系统是否意识到需要 Skill。	人工标注任务-技能需求，记录是否调用 skill_view。
Relevance Precision	被加载的 Skill 是否真正相关。	人工或 LLM judge；结合任务成功率。
Task Success	最终任务是否完成。	单元测试、端到端验收、人工评分。
Tool Efficiency	使用 Skill 后工具调用次数、时间、token 是否下降。	trace 日志。
Regression Safety	Skill patch 后旧任务是否变差。	历史任务集回放。
Skill Pollution Rate	新建/修改内容中有多少是一次性或错误沉淀。	人工审计或规则审计。
Curator Quality	归档/合并是否减少噪声且不损失关键能力。	前后路由准确率、任务成功率、库规模。

23.3 Baseline 与消融

实验组	说明	预期观察
No Skill	只用普通系统提示和工具。	复杂任务成功率低，工具调用更混乱。
Prompt-only Skill	把完整 Skill 直接塞上下文。	短期效果好但上下文成本高，规模化差。
Hermes Metadata Routing	name/description/category + skill_view。	上下文成本低，但 need/relevance awareness 不稳定。
Hybrid Retrieval + Hermes Loading	先 BM25/embedding 召回候选，再 LLM 选择并 skill_view。	路由可观测性和召回率更好。
With Evolution Gate	写回前做回归评测。	污染率下降，但成本上升。
With Curator/Consolidation	周期治理和合并。	库规模下降，重复减少，但需监控误归档。

基于上传材料整合，不代表对最新上游 main 的联网复核

24. 组会/面试讲述模板

下面给出一版可以直接用于组会或面试的三分钟讲述。

三分钟讲述稿

Hermes 的 Skill 子系统可以理解为模型外部的程序性记忆层。它不是把技能存成向量 chunk，而是把一个技能做成目录：入口是 SKILL.md，旁边可以带 references、templates、scripts 和 assets。系统启动时扫描这些目录，只把 name、description、category 这类轻量元数据放进 system prompt。LLM 根据用户任务自行判断是否需要某个 Skill，如果需要，再调用 skill_view 精确加载完整 SKILL.md；复杂场景再按 file_path 加载支持文件，这就是渐进式披露。

它的自进化也不是训练模型参数，而是任务后复盘。主任务结束后，TurnFinalizer 在满足 _skill_nudge_interval 等条件时启动 Background Review。后台会 fork 一个 review_agent，只允许 memory 和 skill 管理工具。它根据对话和工具轨迹判断是否存在可复用经验：例如用户纠正了方法、任务里出现非平凡 debug pattern、已加载 Skill 过时。若值得沉淀，就通过 skill_manage 去 create、patch 或 write_file。写入前有 read-before-write、所有权保护、write approval、security scan 和原子写回。写入后清缓存并记录 usage。长期来看，Curator 根据 .usage.json 把 Skill 从 active 迁移到 stale 或 archived，并可选做 consolidation，把重复微技能合并成 umbrella skill。

所以 Hermes Skill 的核心价值是：用文件夹式能力包替代单段 prompt，用渐进加载控制上下文成本，用后台复盘和生命周期治理让程序性知识库持续演进。它的短板是 need awareness 和 relevance awareness 依赖 LLM 与 description，缺少显式检索分数和回归评测，因此后续研究可以把 hybrid retrieval、Skill Graph 和 evolution gate 接到这套表示层上。

25. 最终专业判断

如果用一句工程化判断概括：Hermes 的创新点不在“训练出了会自我进化的模型”，而在“把 Agent 的可复用经验外化成可审计、可携带、可渐进加载、可治理的文件系统对象”。这使 Skill 既比普通 Prompt 更工程化，也比传统 RAG 更面向行动；但它的路由与演化质量仍高度依赖 description、LLM 判断和后续评测门控。